

Mini proyecto del protocolo ESP NOW

->RIIRLB

Clave Morse entre dos ESP

- Un ESP (el **botón**) envía un mensaje al presionar el botón.
- Otro ESP (el **LED**) recibe el mensaje y enciende un LED.

Para hacer esto, configuraremos uno como **emisor** y otro como **receptor**.

Configuración Básica:

1. **Emisor** (ESP con el botón):
 - Envía un mensaje específico ("ON", "OFF", o el código Morse).
2. **Receptor** (ESP con el LED):
 - Recibe el mensaje y realiza una acción (enciende o apaga el LED).

Conectando 2 ESP con ayuda del profe C.G.P.T.

Pasos para Configurar Ambos Dispositivos:

1. **Obtener las direcciones MAC:**
 - Puedes encontrar la dirección MAC de cada ESP32 ejecutando el siguiente código:

```
import network
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
print(wlan.config('mac'))
```

2. **Subir el código:**
 - Sube el código del emisor a un ESP32.
 - Sube el código del receptor a otro ESP32.
3. **Conectar el hardware:**
 - Conecta un botón al pin GPIO 4 del emisor (configurado como entrada con resistencia pull-up).
 - Conecta un LED al pin GPIO 2 del receptor (configurado como salida).
4. **Prueba:**
 - Presiona el botón en el emisor y verifica que el LED en el receptor se encienda.

Dirección de ESP-32 A: b'\x94<\xc62\xca0'

Dirección de ESP-32 B: b'\x08\xb6\x1f7\xd6\\'

Código para el **Emisor** (ESP con el botón)

```
# Emisor
import espnw
import network
from machine import Pin
from time import sleep

# Configurar WiFi en modo Station
wlan = network.WLAN(network.STA_IF)
wlan.active(True)

# Crear la instancia de ESP-NOW
esp = espnw.ESPNow()
esp.active(True)
peer_mac = b'\x08\xb6\x1f7\xd6\\' # Cambia por la MAC del receptor
esp.add_peer(peer_mac)

# Configurar el botón
button = Pin(4, Pin.IN, Pin.PULL_UP)

estado_anterior = 1 # Inicialmente el botón no está presionado

# Enviar mensajes al presionar o soltar el botón
while True:
    estado_actual = button.value() # Leer estado del botón
    if estado_actual != estado_anterior: # Solo enviar cuando haya un cambio
        if estado_actual == 0:
            print("¡Botón presionado! Enviando señal...")
            esp.send(peer_mac, b"MORSE") # Enviar "MORSE"
        else:
            print("Botón liberado, apagando señal...")
            esp.send(peer_mac, b"0") # Enviar mensaje vacío para apagar el LED

    estado_anterior = estado_actual # Actualizar estado anterior
    sleep(0.01) # Pequeña pausa para evitar rebotes
```

Explicación del Código del Emisor

Este dispositivo tiene un botón que, al presionarlo, envía un mensaje a través de **ESP-NOW**. Cuando el botón es liberado, envía otro mensaje para apagar el LED del receptor.

Explicación paso a paso:

1. **Configuración del WiFi en modo "Station":**

El protocolo ESP-NOW solo funciona si el WiFi está activo en modo **Station (STA)**. Aquí activamos este modo para que el ESP pueda enviar datos.

2. Inicialización de ESP-NOW:

- `espnw.ESPNow()`: Crea una instancia del protocolo ESP-NOW.
- `esp.active(True)`: Habilita ESP-NOW para empezar a usarlo.

3. Registro del receptor (peer):

Necesitamos registrar el receptor usando su dirección MAC. Cambia el valor de `peer_mac` por la dirección MAC del receptor que desees controlar.

Nota: Las direcciones MAC son únicas para cada dispositivo y funcionan como "direcciones postales" en la red.

4. Configuración del botón:

Configuramos el pin GPIO 4 como entrada con resistencia **pull-up** para evitar señales inestables cuando el botón no está presionado.

5. Detección de cambios en el botón:

Detectamos si el estado del botón (presionado/liberado) ha cambiado. Esto asegura que solo enviamos un mensaje al detectar un cambio, evitando sobrecargar la comunicación.

6. Envía mensajes según el estado del botón:

- `esp.send()`: Envía el mensaje al receptor registrado en `peer_mac`.
- Si el botón está presionado, se envía "MORSE" para encender el LED. Si se libera, se envía "0" para apagar el LED.

7. Pequeña pausa para evitar rebotes:

Cuando presionas un botón físico, se producen rebotes eléctricos que pueden generar lecturas inestables. Esta pequeña pausa evita lecturas redundantes.

Código para el **Receptor** (ESP con el LED)

```
# Receptor
import espnow
import network
from machine import Pin

# Configurar WiFi en modo Station
wlan = network.WLAN(network.STA_IF)
wlan.active(True)

# Crear la instancia de ESP-NOW
esp = espnow.ESPNow()
esp.active(True)

# Configurar LED en el pin 2 (ajusta según tu hardware)
led = Pin(2, Pin.OUT)
led.value(0) # Apagar LED inicialmente

print("Esperando mensajes...")

# Escuchar mensajes entrantes
```

```
while True:
    host, msg = esp.recv() # Recibe datos (host = MAC del emisor, msg = mensaje)
    if msg:
        mensaje = msg.decode()
        print(f"Mensaje recibido de {host}: {mensaje}")

    if mensaje == "MORSE":
        led.value(1) # Encender LED (botón presionado)
    else:
        led.value(0) # Apagar LED (botón liberado)
```

Explicación del Código del Receptor

Este dispositivo recibe el mensaje enviado por el emisor y enciende/apaga un LED según el mensaje recibido.

Explicación paso a paso:

1. **Configuración del WiFi en modo "Station":** Igual que en el emisor:

Activa el WiFi en modo **Station (STA)**, necesario para usar ESP-NOW.

2. **Inicialización de ESP-NOW:**

Igual que en el emisor, se inicializa ESP-NOW para permitir la recepción de mensajes.

3. **Configuración del LED:**

- Configuramos el pin GPIO 2 como salida para controlar el LED.
- Inicialmente, el LED está apagado (led.value(0)).

4. **Recepción de mensajes:**

- esp.recv(): Escucha continuamente los mensajes entrantes.
- host: Es la dirección MAC del emisor que envió el mensaje.
- msg: Es el mensaje recibido.

5. **Acciones según el mensaje recibido:**

- Si el mensaje es "MORSE", encendemos el LED (el botón está presionado).
- Si el mensaje es "0", apagamos el LED (el botón fue liberado).

¿Cómo interactúan estos códigos?

1. Emisor (ESP con el botón):

- Cada vez que presionas o sueltas el botón, envía un mensaje al receptor:
 - **Botón presionado:** "MORSE".
 - **Botón liberado:** "0".

2. Receptor (ESP con el LED):

- Escucha constantemente los mensajes enviados por el emisor.
- Según el mensaje recibido, decide si encender o apagar el LED:
 - **Mensaje "MORSE":** Enciende el LED.
 - **Mensaje "0":** Apaga el LED.

Conexión Física:

1. Emisor:

- Conecta un botón al pin GPIO 4 y al GND.
- La resistencia pull-up está activada en el código, no necesitas un resistor externo.

2. Receptor:

- Conecta el LED al GPIO 2 con una resistencia de 220Ω a $1K\Omega$ en serie.
- El cátodo del LED (lado corto) va al GND.